

Assignment 1

Task 1

Environment

In this task, our experimental environment is MacOS 14.6.1 (Core: Apple M3 Pro, Memory:18GB)

Compile files and generate data (optional)

1.Monet DB

First, we go to the `../dbgen/` directory and execute the compile command `make -f makefile.suite.MonetDB`. Since Mac does not have `malloc.h`, an error message will be displayed: `malloc.h` file not found. For the complete error message, see figure 1.

```
bm_utils.c:71:10: fatal error: 'malloc.h' file not found
#include <malloc.h>
      ^~~~~~
1 error generated.
```

Figure 1: error example

In order to solve this issue, we need to find the line `#include <malloc.h>` in the `bm_utils.c` file and change it to `#include <sys/malloc.h>`. And then re-execute the compile command and encounter the next error, because the `tpcd.h` file does not define the variables that are suitable for generating monetdb statements. You need to add the following content to the file:

```

#ifdef MONETDB
#define GEN_QUERY_PLAN "EXPLAIN SELECT * FROM "
#define START_TRAN "START TRANSACTION;"
#define END_TRAN "COMMIT;"
#define SET_OUTPUT ""
#define SET_ROWCOUNT "LIMIT %d;"
#define SET_DBASE ""
#endif

```

Figure 2: define variables

Then, we continue to execute the compilation command. When we encounter the last error, there is no <malloc.h> in the varsub.c file. Just change it to #include <sys/malloc.h> as above. And then re-execute the compilation command. The compilation is successful and dbgen and qgen are generated..

We enter the ../dbgen/ directory and execute ./dbgen.sh 1 to generate sf1 data. To generate data of other sizes, replace the number at the end, ./dbgen.sh <size>.

2.PostgreSQL

Since the second database used is postgresql, it does not support delimiters at the end of each row of data when importing data, so I wrote a script to process the generated data and remove the delimiters at the end of each row. The file is process_files.sh. (Put this file in the generated data directory. After execution, a data cleaned folder will be generated, which contains the processed files.)

We enter the ../dbgen/ directory and execute ./dbgen.sh 1 to generate sf1 data. To generate data of other sizes, replace the number at the end, ./dbgen.sh <size>.

Set Up for Monet DB and PostgreSQL

Install Monet DB / PostgreSQL by homebrew.

1.Monet DB

1. brew install monetdb
2. monetdbd create /monetdb/mydbfarm // Create a workspace
3. monetdbd start /monetdb/mydbfarm
4. monetdb create sf1 //create database
5. mclient -u monetdb -d sf1

ps: Create a .monetdb file with the following content::

```

user=<replace your username>
password=<replace your password>

```

Create an environment variable DOTMONETDBFILE to point to the .monetdb file you just created. You don't need to enter your password when you use the mclient command later.

2.PostgreSQL

1. brew install postgres // my version is V14.13
2. psql postgres //the default database is postgres
3. create database sf1

\c sf1 // enter the created sf1 database

Load Data

Monet DB can directly use the provided SQL to execute the SQL step by step to create tables, import data, and create keys.

The statements used for PostgreSQL import are slightly different and need to be slightly modified. The following is an example:

copy region from '<replace with your path> /region.tbl' delimiter '|';

Run SQL

Due to syntax differences, q1.sql cannot be run directly in postgresQL, so it has been slightly adjusted. See q1.sql. At the same time, in order to facilitate comparison of output results, \pset footer off is added before each postgresQL sql statement to remove the number of output lines at the end. q17.sql and q20.sql have not run out of results, so these two are skipped

In order to facilitate batch running, all SQLs are put together in a queries directory, two scripts run_queries_<monetdb/postgre>.sh and get_time_<monetdb/postgre>.sh

Usage :

./run_queries_<monetdb/postgre>.sh <replace with your database>

./get_time_<monetdb/postgre>.sh <replace with your database> [execution_count]

(PS:execution_count is the number of times each SQL is executed, default value is 1)

run_queries_<monetdb/postgre>.sh will generate a result/monetdb/<database name>directory, all execution results are named, for example, 1.out and then placed in this folder

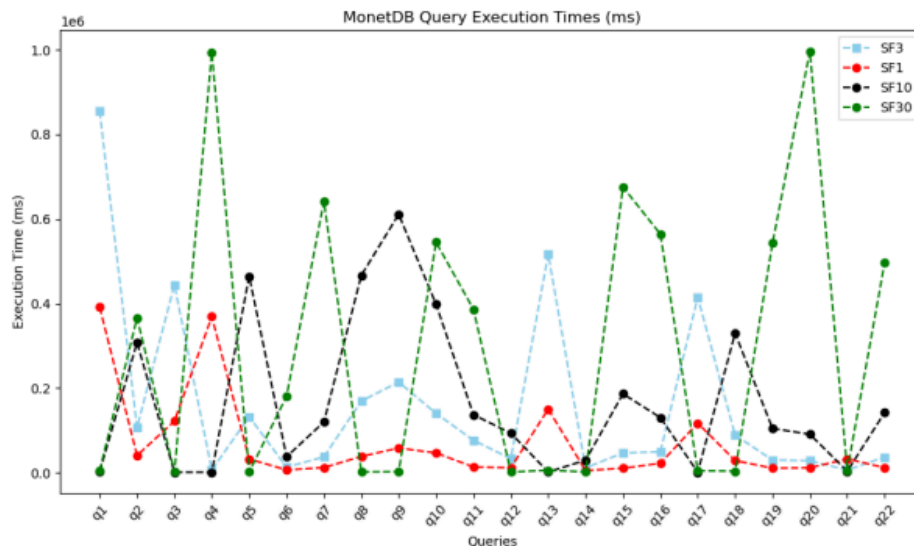
get_time_<monetdb/postgre>.sh will output the running time of each SQL in turn.

Verify the results

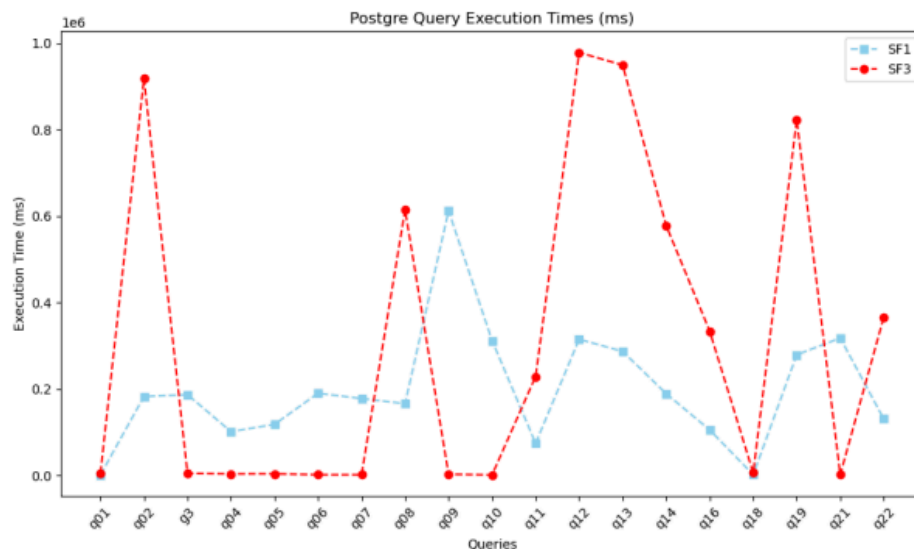
Put the sf1 folder and answers folder containing the running results into the check_answers folder and run ./pairs.sh sf1 answers in the ../check_answers

directory. A sf1_answers folder will be output, which contains a folder such as analysis_1.log, which contains the number of differences and difference details. Found 0 unacceptable mismatches.

In the graphs below, the first graph shows the time taken by MonetDB to execute queries across four different data sizes, while the second graph illustrates PostgreSQL's execution times for two data sizes. As the data size increases, both database management systems require more time, and the increase in time is quite significant.



Graph 1 : Time taken by MonetDB



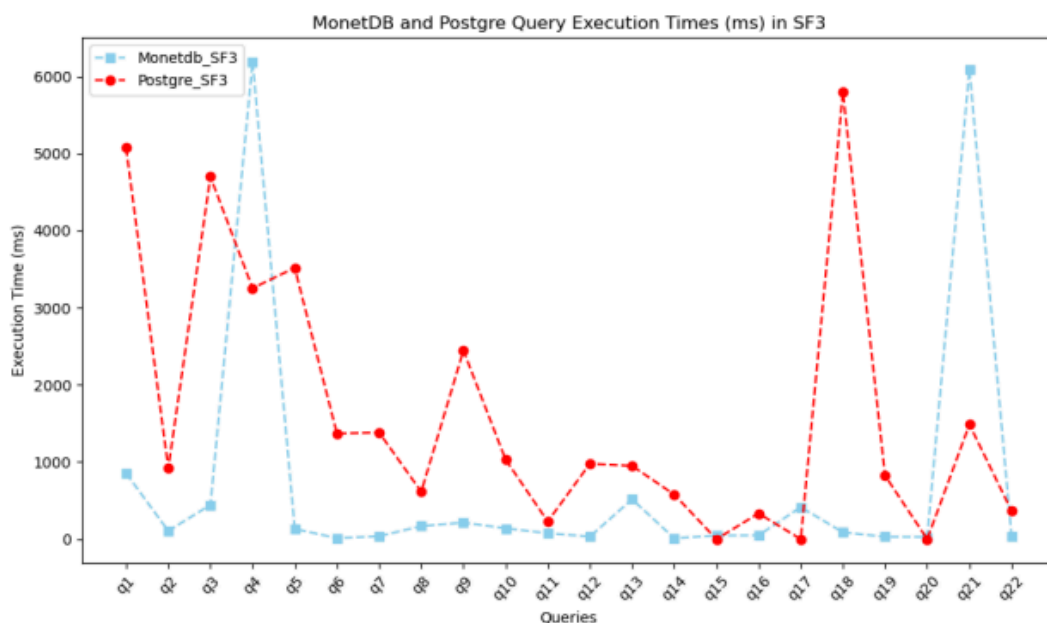
Graph 2 : Time taken by Postgre SQL

In the graphs below, we compare the query execution times of two database management systems executing the same SQL queries on identical data sizes. In both the first and second graphs, MonetDB generally outperforms PostgreSQL, except for a few specific SQL scripts. Several factors can explain these differences:

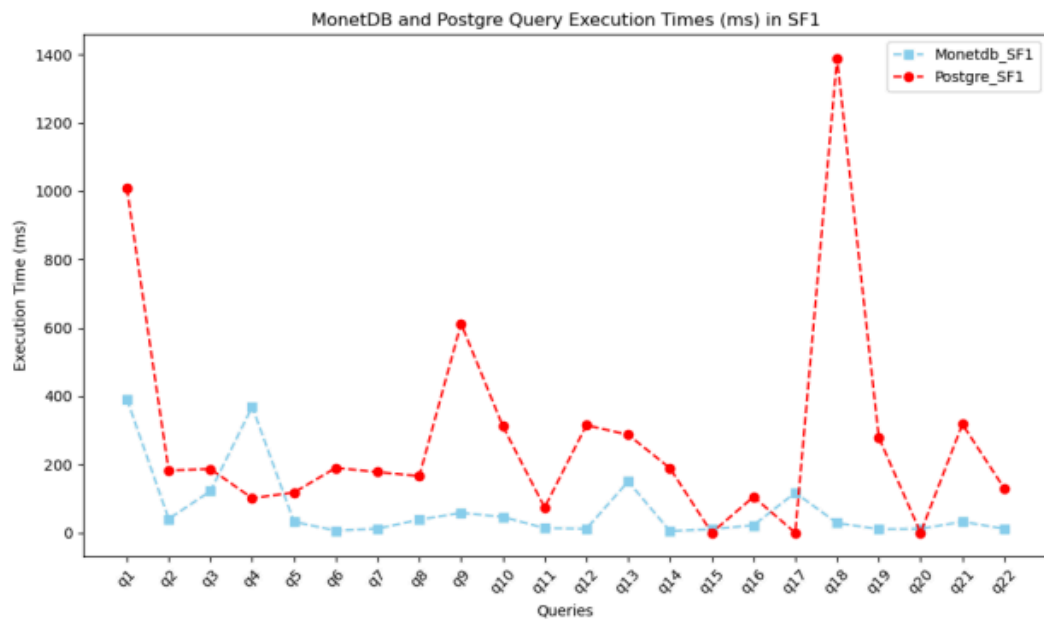
- MonetDB: uses a column storage format, which is particularly suitable for processing analytical workloads. Column storage can compress data more efficiently and access required data faster when executing complex queries.

PostgreSQL: uses a traditional row storage format, which is more flexible, but may not perform as well as column storage when processing large-scale data analysis.

- MonetDB: Uses vectorized execution and parallel processing techniques that can significantly improve query performance, especially when processing large data sets. PostgreSQL: Although there are some optimization techniques, it is mainly designed for transactional workloads and may not perform as well as MonetDB1 when processing complex analytical queries.
- MonetDB: uses bitmap indexes and compressed column storage to optimize query performance. PostgreSQL: provides a variety of index types and data structures for different usage scenarios, but may not be as efficient as MonetDB in some analytical queries



Graph 3 : Query Execution Times(ms) in SF3



Graph 4: Query Execution Times(ms) in SF1

Additional Benchmark

1.Hardware Info

In this part we use ROG ZEPHYRUS G16 and ROG ALLY game console to run SF3 as a comparison. The details of two hardware platform are as follows:

ROG ZEPHYRUS G16 2024:

CPU: Intel Ultra 9 185H 6P8E Cores, 2.3GHz@P core, 1.8Ghz @E core, Cache L1:112 KB (per core),Cache L2: 2 MB (per core), Cache L3: 24 MB (shared), E-Core L1: 96 KB (per core), E-Core L2: 2 MB (per module)

Memory: 32GB@7467MT/s

Windows

TPC-H(sf=3)

MonetDB v11.51.3 (Aug2024)

ROG ALLY game console:

CPU: AMD Z1 EXTREME 8C/16T, 3.3GHz, Cache L1: 64KB, Cache L2: 1MB, Cache L3: 16MB

Memory: 11.7GB@6400MT/s

Windows

TPC-H(sf=3)

MonetDB v11.51.3 (Aug2024)

2.Benchmark Data

The data are as follows: The first row of every single query is a cold start, The 2-3 rows of every query are hot starts, aiming for the comparison.

SF3 (ROG ZEPHYRUS G16)					
query	sql	opt	run	clk(ms)	PS
1	1.999	12.786	1156.508	1158.117	
1	0	14.599	271.578	272.709	
1	0	16.513	283.325	284.301	
2	0	5.31	334.232	334.701	
2	0.508	4.006	19.156	19.423	
2	1	6.517	23.134	23.937	

3	0	1.999	488.596	487.883	
3	1.007	1.998	86.372	86.433	
3	3	3	67.51	68.289	
4	0	3	1013.575	1014.226	
4	0	1.01	39.07	39.323	
4	0	0	41.065	41.197	
5	1.001	1.999	138.118	138.614	
5	0	2.001	64.736	65.226	
5	0	1.001	68.104	67.726	
6	0	7	42.125	41.619	
6	0	4.133	20.27	20.38	
6	0	5.001	23.382	24.334	
7	2.123	4.974	87.815	87.18	
7	0.999	2.006	61.533	61.725	
7	0	3.001	56.967	57.718	
8	0	1.997	142.736	143.076	
8	0	2.996	92.983	93.549	
8	1.005	1.994	100.596	101.84	
9	0	1.472	252.143	252.55	
9	0	1.996	82.199	82.61	
9	0.999	2.108	80.585	81.32	
10	0	0.997	316.709	320.431	
10	0	2	57.732	58.326	
10	0	2.461	56.868	57.457	
11	1.01	1.015	61.57	61.418	
11	0.504	0.503	10.565	11.601	

11	0	1.365	13.438	13.348	
12	0	3.002	56.6	56.393	
12	0	1.142	29.589	29.726	
12	0	3	31.69	31.558	
13	0	0	731.131	730.932	
13	0	1	258.865	259.84	
13	1.021	0	269.478	269.922	
14	0	0	0	54.634	ERROR = loverflow in conversion to DECIMAL(18,10). CODE = 22003
14	0	0	0	16.604	ERROR = loverflow in conversion to DECIMAL(18,10). CODE = 22003
14	0	0	0	17.74	ERROR = loverflow in conversion to DECIMAL(18,10). CODE = 22003
15	0 0 0	0 2.52 3 0	14.7 50.191 1 .142	21.517 51.486 2.275	
15	0 0 0	0 1.00 7 0	0.664 28.582 0	4.582 29.029 1. 566	
15	0 0.99 9 0	0 2.00 3 0	1.132 31.532 0	4.333 31.481 2. 253	
16	0	1	98.468	99.323	
16	0.506	1.006	77.389	78.383	
16	0	1.128	73.676	75.227	
17	1.056	1.139	96.331	95.967	
17	0	1.998	77.095	77.987	
17	0	1.139	75.357	76.073	
18	0	1.517	117	118.538	

18	0	0	91.011	91.866	
18	0	1.005	87.312	88.233	
19	0	2.512	32.423	32.964	
19	0	2.696	29.752	30.316	
19	0	1.999	28.107	26.891	
20	1.003	1.998	65.203	66.456	
20	0	2.011	32.439	32.414	
20	0	0.998	33.926	34.842	
21	0	13.144	173.24	173.713	
21	0	21.84	128.433	128.008	
21	0	15.353	111.982	112.684	
22	0	1.011	74.225	73.742	
22	0	0	57.603	57.739	
22	0	0	51.422	51.977	

Form1. SF3 in laptop

SF3 (ROG ALLY)					
query	sql	opt	run	clk(ms)	PS
1	1.532	6.556	399.112	399.814	
1	0	6.516	298.293	299.938	
1	0	11.038	289.273	291.11	
2	1.107	9.301	91.558	92.716	
2	0	2.01	15.223	15.464	
2	0	2.004	16.519	16.788	
3	0	1.019	164.6	165.313	
3	0	2.306	80.914	81.322	
3	0	2.005	69.814	69.794	

4	0	0.525	102.789	103.705	
4	0	0	52.574	52.587	
4	0	1.004	54.04	53.427	
5	0	1.002	97.467	97.273	
5	0	2.006	67.913	68.227	
5	0	1.224	65.258	65.367	
6	0.53	2.366	45.415	45.394	
6	0	2.054	19.64	19.999	
6	0	2.005	20.639	20.105	
7	0.782	2.223	81.175	81.792	
7	0	1.509	55.038	55.009	
7	0	2.004	55.991	56.432	
8	1.439	1.203	146.17	146.146	
8	1.002	2.006	109.352	110.096	
8	1.002	2.094	111.868	112.717	
9	1.006	1.002	133.722	134.135	
9	0	2.004	79.007	79.794	
9	0	2.008	82.837	82.613	
10	0	2.04	330.283	332.529	
10	0	2.002	65.894	65.447	
10	0	2.005	59.15	59.786	
11	0	2.004	37.135	37.505	
11	0	2.445	18.143	18.361	
11	0	1.396	15.702	16.029	
12	0	1.083	53.136	53.264	
12	0	2.002	33.496	33.025	

12	0	1.003	32.752	33.066	
13	0	1.003	228.459	228.533	
13	0	0	179.92	180.063	
13	0	1.327	180.812	180.894	
14	0	0	0	46.042	ERROR = !overflow in conversion to DECIMAL(18,10). CODE = 22003
14	0	0	0	15.925	ERROR = !overflow in conversion to DECIMAL(18,10). CODE = 22003
14	0	0	0	16.232	ERROR = !overflow in conversion to DECIMAL(18,10). CODE = 22003
15	0 0 0	0 1.509 0	12.629 38.156 1 .004	14.965 38.533 2 .112	
15	0 0 0	0 1.003 0	1.002 30.675 0	1.825 31.021 1. 243	
15	0 0 0	0 1.021 1.038	1.026 30.887 1. 038	1.810 30.127 1. 670	
16	0	1.003	90.822	93.549	
16	0	1.002	75.602	78.293	
16	1.417	1.504	75.949	79.016	
17	0	1.003	71.141	71.523	
17	0	2.005	15.154	14.891	
17	0	2.081	9.502	8.982	
18	0	1.002	124.278	123.862	
18	0	1.003	100.949	100.551	
18	0	1.002	106.97	107.113	
19	0	1.001	37.738	38.722	

19	1.002	1.002	29.914	29.719	
19	0	2.11	31.912	31.814	
20	0	1.003	57.016	56.849	
20	0	0.507	33.047	33.881	
20	1.002	1.017	36.371	36.546	
21	1.002	8.021	161.751	162.683	
21	0	8.03	140.319	140.828	
21	0	7.243	134.216	135.062	
22	0	1.507	63.154	62.558	
22	0	1.059	39.826	39.969	
22	0	0	40.662	41.278	

Form 2. SF3 in game console

3. Questions & Speculation

The biggest question of the two forms is the comparison of the price and performance. ROG ZEPHYRUS G16(laptop as follows) has much more expensive price than ROG ALLY(game console as follows), even the chip price of ultra 9 shows in Intel is equal to the whole price of the game console. However, the benchmark shows the similar results as above. Here we have two speculations:

1.The mode when we ran the benchmark is single thread mode, which is at similar performance even that of game console is stronger than the laptop in CPU.

2.The results shown in the terminal need time. The command we use is to get the performance shown in the terminal directly. Some differences such as screen resolution may cause the result. The screen of the laptop(2k 165Hz) needs more time than the game console(1080p 120Hz) to show the result, which may offset the time difference.

Task 2

Python implement

We choose python and use the panda library to implement Q1 and Q6, and perform database-like operations. First, We can use this command to generate the lineitem.csv file from monetdb, then import it into python file for further processing.

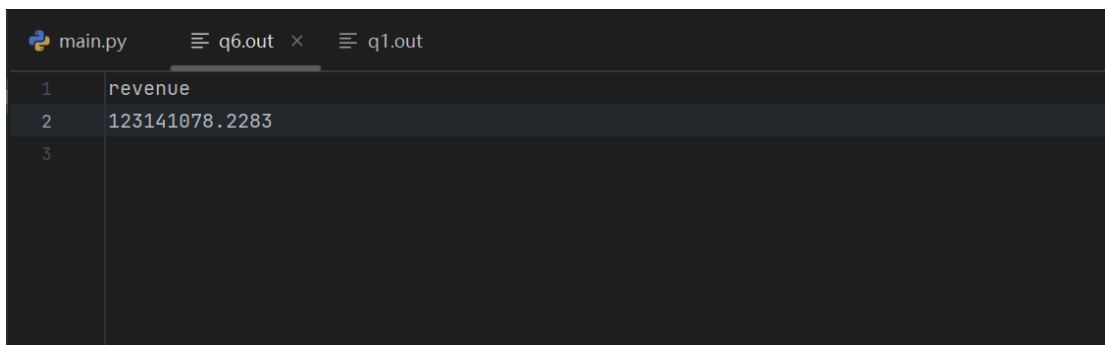
Command: `mclient -d sf1 -f csv+"," -s "select * from lineitem;" > ./lineitem.csv`.

First implement Q6, record the query start time, and then filter the imported data to obtain the data that meets the conditions. And calculate the total income.

Then implement Q1, also record the query start time first, filter the data, and group by `I_returnflag` and `I_linestatus` according to the requirements of the question and perform aggregate statistics. Group by `I_returnflag` and `I_linestatus` and perform aggregate statistics. Finally, output the running time and results of Q6 and Q1.

Verify the results

In the monetdb database, we use SQL to query Q1 and Q6 in the same way, and then compare the results output by Python with the results in the database. From the results in the following figures, we can see that the data processing results are consistent.



1	revenue
2	123141078.2283
3	

Figure 3: Q6.out(MonetDb)

```
C:\anaconda\python.exe "C:\course\data management\Assignment 0\ADM_assignment1_V0\main.py"
Q6 result total Revenue: 123141078.22829999
Q6 Filtering and calculation time: 0.1617 seconds
Q1 Filtering and calculation time: 3.4553 seconds
Q1 result
  l_returnflag l_linestatus      sum_qty  ...      avg_price  avg_disc  count_order
0            A            F  37734107.0  ...    38273.129735  0.049985    1478493
1            N            F   991417.0  ...    38284.467761  0.050093     38854
2            N            O  74476040.0  ...    38249.117989  0.049997    2920374
3            R            F  37719753.0  ...    38250.854626  0.050009    1478870
```

Figure 4: Q6.out(python)

```
main.py  q6.out  q1.out x
1  l_returnflag|l_linestatus|sum_qty|sum_base_price|sum_disc_price|sum_charge|avg_qty|avg_price|avg_disc|count_order
2  A|F|37734107.00|56586554400.73|53758257134.8700|55909065222.827692|25.52|38273.13|0.05|1478493
3  N|F|991417.00|1487504710.38|1413082168.0541|1469649223.194375|25.52|38284.47|0.05|38854
4  N|O|74476040.00|111701729697.74|106118230307.6056|110367043872.497010|25.50|38249.12|0.05|2920374
5  R|F|37719753.00|56568041380.90|53741292684.6040|55889619119.831932|25.51|38250.85|0.05|1478870
6
```

Figure 5: Q1.out(MonetDb)

```
Q1 Filtering and calculation time: 3.4553 seconds
Q1 result
  l_returnflag l_linestatus      sum_qty  ...      avg_price  avg_disc  count_order
0            A            F  37734107.0  ...    38273.129735  0.049985    1478493
1            N            F   991417.0  ...    38284.467761  0.050093     38854
2            N            O  74476040.0  ...    38249.117989  0.049997    2920374
3            R            F  37719753.0  ...    38250.854626  0.050009    1478870

[4 rows x 10 columns]
```

Figure 6: Q1.out(MonetDb)

Compare execution time

The following figures shows that under SF1, monetdb takes 391.331ms and 6.304ms to execute Q1 and Q6 respectively, while postgre takes 1008.800ms and 189.977ms to execute Q1 and Q6 respectively. But our implementation in python takes 3455.3ms and 161.7ms to execute Q1 and Q6. We can see that monetdb is the most efficient way. And our implementation has the least efficient in compare.

```
C:\anaconda\python.exe "C:\course\data management\Assignment 0\ADM_assignment1_V0\main.py"
Q6 result total Revenue: 123141078.22829999
Q6 Filtering and calculation time: 0.1617 seconds
Q1 Filtering and calculation time: 3.4553 seconds
Q1 result
  l_returnflag l_linestatus      sum_qty  ...      avg_price  avg_disc  count_order
0            A            F  37734107.0  ...    38273.129735  0.049985    1478493
1            N            F   991417.0  ...    38284.467761  0.050093     38854
2            N            O  74476040.0  ...    38249.117989  0.049997    2920374
3            R            F  37719753.0  ...    38250.854626  0.050009    1478870
```

Figure 7: execution time in python

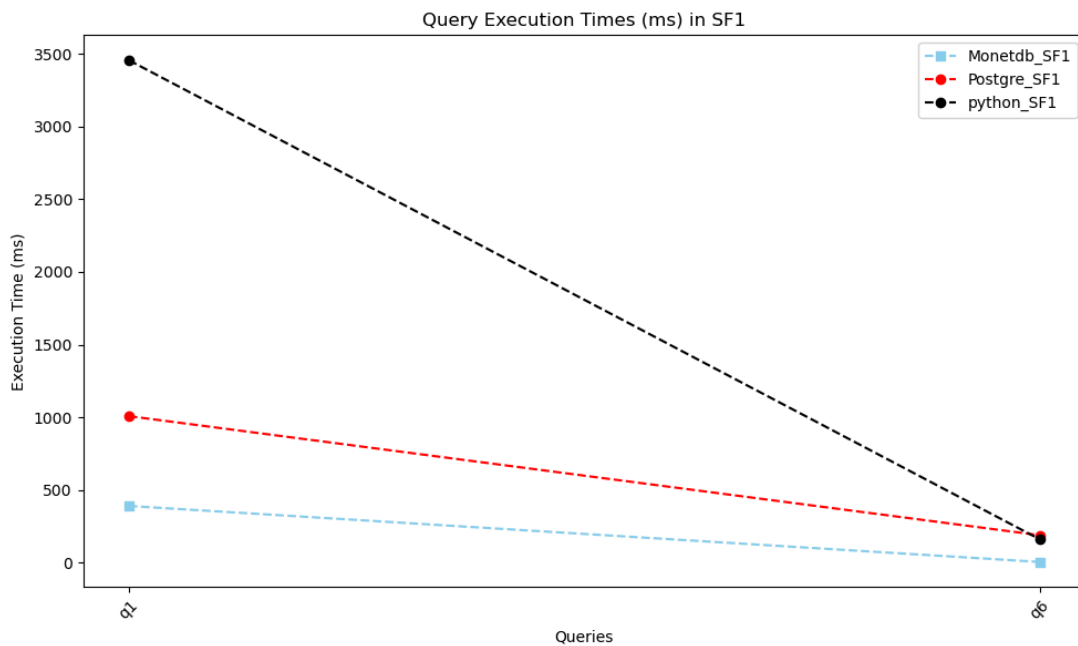


Figure 8: compare of three results

Own implementation compares to DBMS(s)

Database management systems like MonetDB are specifically designed for efficient querying of large-scale data sets. It uses a variety of optimization techniques, including storage format optimization, parallel processing technology, vectorized execution, and multiple index types, to enable it to execute queries in an optimal way. Python's pandas library is very powerful, but pandas usually performs operations in memory and lacks the sophisticated query optimization strategies of DBMS. Therefore, when processing complex queries or large data sets, Python's execution speed is often not as fast as DBMS.